

Capacitated Vehicle Routing Problem

Soluzione tramite Algoritmo Genetico Ibrido (HGA)

Relazione di Progetto

Contents

1	Introduzione al Problema	3
2	Scelta dell'Algoritmo	3
2.1	Motivazione	3
2.2	Alternative Considerate	4
3	Progettazione dell'Algoritmo	4
3.1	Encoding e Decodifica	4
3.2	Algoritmo di Split	4
3.3	Popolazione Iniziale	5
3.4	Selezione	5
3.5	Crossover: Order Crossover (OX)	5
3.6	Mutazione	6
3.7	Ricerca Locale	6
3.8	Elitismo e Rimpiazzo	7
3.9	Criterio di Arresto	7
3.10	Parametri Riepilogativi	7
4	Protocollo Sperimentale	7
4.1	Istanze	7
4.2	Setup Sperimentale	8
5	Risultati Sperimentali	8
5.1	Risultati Complessivi	8
5.2	Analisi della Convergenza	9
5.3	Visualizzazione delle Rotte Migliori	9
5.4	Analisi Comparativa Generale	10
5.5	Discussione dei Risultati	11
6	Scelte Progettuali e Originalità	12
6.1	Scelte Chiave	12
6.2	Difficoltà Affrontate	12
6.3	Elementi di Originalità	12

1 Introduzione al Problema

Il **Capacitated Vehicle Routing Problem (CVRP)** è una delle varianti più studiate del classico Vehicle Routing Problem (VRP). Si tratta di un problema di ottimizzazione combinatoria NP-hard che consiste nel determinare un insieme di percorsi di costo minimo per una flotta di veicoli, al fine di servire un dato insieme di clienti geograficamente distribuiti, partendo e ritornando ad un deposito centrale.

Formalmente, sia dato un grafo completo $G = (V, E)$, dove:

- $V = \{v_0, v_1, \dots, v_n\}$ è l'insieme dei vertici, con v_0 che rappresenta il *depot* e $V \setminus \{v_0\}$ l'insieme dei clienti;
- E è l'insieme degli archi, ad ognuno dei quali è associato un costo $d_{ij} \geq 0$ (distanza euclidea tra i nodi i e j).

Ad ogni cliente $i \in V \setminus \{v_0\}$ è associata una domanda $q_i \geq 0$, e sono disponibili m veicoli, ciascuno con capacità σ . L'obiettivo è determinare per ogni veicolo un itinerario tale che:

- (i) Ogni cliente sia visitato esattamente una volta da un solo veicolo;
- (ii) Ogni percorso inizi e termini al depot;
- (iii) La somma delle domande dei clienti assegnati a ciascun veicolo non superi la capacità σ .

L'obiettivo è minimizzare il costo totale:

$$\min \sum_{h=1}^m \sum_{(i,j) \in \eta_h} d_{ij} \quad (1)$$

dove η_h è l'insieme degli archi percorsi dal veicolo h .

2 Scelta dell'Algoritmo

2.1 Motivazione

Tra gli algoritmi proposti per il progetto, la scelta è ricaduta sull'**Algoritmo Genetico Ibrido (HGA)**. Le motivazioni sono le seguenti:

- 1. Esplorazione globale efficace:** Gli algoritmi genetici esplorano efficientemente lo spazio delle soluzioni attraverso i meccanismi di selezione, crossover e mutazione, riducendo il rischio di convergenza prematura verso minimi locali.
- 2. Raffinamento locale:** L'ibridazione con operatori di ricerca locale (2-opt, Or-opt, Relocate, Exchange) permette di migliorare le soluzioni prodotte dal processo evolutivo, combinando l'esplorazione globale con lo sfruttamento locale.
- 3. Flessibilità di rappresentazione:** L'encoding basato su permutazione, accoppiato con l'algoritmo di split di Prins, permette di rappresentare e valutare efficientemente soluzioni con numero variabile di veicoli.

4. **Risultati allo stato dell'arte:** L'HGA è riconosciuto in letteratura come uno degli approcci più efficaci per il CVRP, producendo risultati competitivi su istanze benchmark standard.
5. **Robustezza:** La combinazione di operatori diversi garantisce robustezza su diverse tipologie di istanze (set A, B, E, P con caratteristiche differenti).

2.2 Alternative Considerate

- **Ant Colony Optimization (ACO):** Sebbene naturalmente adatto a problemi di routing, l'ACO richiede una calibrazione molto fine dei parametri (feromoni, evaporazione, euristica) e può soffrire di convergenza prematura.
- **Algoritmo Immunologico:** Interessante dal punto di vista teorico, ma con meno evidenze empiriche di efficacia sul CVRP rispetto all'HGA.

3 Progettazione dell'Algoritmo

3.1 Encoding e Decodifica

L'algoritmo utilizza un **encoding a permutazione** dei clienti (escluso il depot). Ogni cromosoma è una permutazione $\pi = (\pi_1, \pi_2, \dots, \pi_n)$ dei nodi $1, 2, \dots, n$. La decodifica da permutazione a soluzione (insieme di route) è effettuata tramite l'**algoritmo di Split** di Prins (2004).

3.2 Algoritmo di Split

L'algoritmo di Split trasforma una permutazione in un insieme ottimale di route che rispettano i vincoli di capacità, utilizzando programmazione dinamica.

Sia $\pi = (\pi_1, \dots, \pi_n)$ una permutazione di clienti. Definiamo $V[k]$ come il costo minimo per servire i primi k clienti della permutazione. L'algoritmo procede come segue:

Algorithm 1 Split Algorithm (Prins, 2004)

```
1:  $V[0] \leftarrow 0$ 
2:  $P[0] \leftarrow 0$ 
3: for  $i \leftarrow 1$  to  $n$  do
4:    $V[i] \leftarrow \infty$ 
5:   load  $\leftarrow 0$ 
6:   for  $j \leftarrow i$  down to 1 do
7:     load  $\leftarrow$  load + demand[ $\pi_j$ ]
8:     if load > capacity then
9:       break
10:    end if
11:    routeCost  $\leftarrow d_{0,\pi_j} + \sum_{k=j}^{i-1} d_{\pi_k,\pi_{k+1}} + d_{\pi_i,0}$ 
12:    newCost  $\leftarrow V[j-1] +$  routeCost
13:    if newCost <  $V[i]$  then
14:       $V[i] \leftarrow$  newCost
15:       $P[i] \leftarrow j - 1$ 
16:    end if
17:  end for
18: end for
```

La complessità è $O(n^2)$, efficiente anche per le istanze più grandi considerate ($n \leq 101$). Dopo la DP, si ricostruiscono le route tramite backtracking da $P[n]$.

3.3 Popolazione Iniziale

La popolazione iniziale ($\mu = 100$ individui) è generata combinando:

1. **Nearest Neighbor Heuristic:** Costruisce una permutazione partendo dal depot e selezionando iterativamente il cliente più vicino non ancora visitato.
2. **Clarke-Wright Savings Heuristic:** Utilizza l'algoritmo dei risparmi per costruire route greedy, poi le appiattisce in una permutazione.
3. **Permutazioni casuali:** Le restanti permutazioni sono generate in modo random per garantire diversità genetica.

3.4 Selezione

Viene utilizzata la **selezione a torneo** con $k = 2$. Due individui sono scelti casualmente dalla popolazione e il migliore (costo minore) viene selezionato. Questo metodo offre un buon bilanciamento tra pressione selettiva e diversità.

3.5 Crossover: Order Crossover (OX)

L'Order Crossover preserva l'ordine relativo dei geni da entrambi i genitori. Dati due genitori P_1 e P_2 :

1. Si selezionano due punti di taglio casuali.
2. Il segmento compreso tra i due punti viene copiato da P_1 al figlio.

3. Le posizioni rimanenti sono riempite con i geni di P_2 nell'ordine in cui appaiono, saltando quelli già presenti.

Questo operatore garantisce che il figlio sia una permutazione valida e mantenga strutture parziali da entrambi i genitori. Il tasso di crossover è $p_c = 0.9$.

3.6 Mutazione

Sono implementati tre operatori di mutazione, scelti casualmente con probabilità differenziate:

1. **Swap Mutation** (40%): Scambia due elementi in posizioni casuali. Operatore semplice che favorisce piccole perturbazioni.
2. **Insert Mutation** (30%): Rimuove un elemento e lo reinserisce in una posizione diversa. Utile per modificare l'ordine relativo.
3. **Inversion Mutation** (30%): Inverte un sottosegmento della permutazione. Efficace per modificare la struttura delle route quando combinato con lo Split.

Il tasso di mutazione è $p_m = 0.3$.

3.7 Ricerca Locale

L'ibridazione è realizzata attraverso quattro operatori di ricerca locale, applicati con probabilità $p_{ls} = 0.1$ ai nuovi individui:

1. **2-opt (intra-route)**: Per ogni route, cerca coppie di archi (a, b) e (c, d) tali che la loro sostituzione con (a, c) e (b, d) riduca il costo totale. Complessità $O(k^2)$ per route di lunghezza k .
2. **Or-opt (intra-route)**: Trasferisce segmenti di 1, 2 o 3 nodi consecutivi in posizioni diverse all'interno della stessa route. Più flessibile del 2-opt per piccoli aggiustamenti.
3. **Relocate (inter-route)**: Sposta un cliente da una route all'altra, provando tutte le posizioni di inserimento. Verifica i vincoli di capacità prima dell'inserimento.
4. **Exchange (inter-route)**: Scambia due clienti appartenenti a route diverse. Controlla che entrambe le route rimangano feasible dopo lo scambio.

La ricerca locale conta come 5 valutazioni di fitness a causa del suo costo computazionale.

Per mitigare l'elevato costo computazionale degli operatori inter-route (Relocate ed Exchange), che richiederebbero altrimenti una scansione completa di complessità $O(N^2)$ per verificare tutte le combinazioni di inserimento e scambio, è stata integrata la **Ricerca Locale Granulare** (Granular Local Search, GLS).

Per ciascun cliente $i \in V \setminus \{v_0\}$ si definisce un intorno di vicinanza Γ_i composto dai soli γ (parametro `granular_size`) clienti più vicini secondo la matrice delle distanze. Durante la ricerca locale:

- Un cliente i viene inserito (Relocate) all'interno di una rotta destinataria solo in una posizione immediatamente adiacente a un nodo j tale che $j \in \Gamma_i$ (o adiacente al depot);
- Due clienti i e j vengono valutati per uno scambio (Exchange) solo se $i \in \Gamma_j$ oppure $j \in \Gamma_i$.

Questo filtro riduce la complessità computazionale dell'esplorazione del vicinato inter-route a $O(N \cdot \gamma)$. Con impostazioni del parametro $\gamma \ll N$ (ad esempio $\gamma = 15$ consigliato, o $\gamma = 3$ per test ultra-rapidi), la GLS consente di eliminare fino al 90% dei calcoli ridondanti per mosse distanti nello spazio, determinando il massivo aumento delle valutazioni al secondo dell'algoritmo.

3.8 Elitismo e Rimpiazzo

I migliori $e = 2$ individui della popolazione corrente sono preservati nella generazione successiva (elitismo). Il resto della popolazione è sostituito dai nuovi individui generati tramite crossover, mutazione e ricerca locale.

3.9 Criterio di Arresto

L'algoritmo si arresta dopo $FE = 3.5 \times 10^5 = 350,000$ valutazioni della funzione fitness, come richiesto dal protocollo sperimentale.

3.10 Parametri Riepilogativi

Table 1: Parametri dell'HGA

Parametro	Valore
Dimensione popolazione (μ)	100
Crossover rate (p_c)	0.9
Mutation rate (p_m)	0.3
Local search rate (p_{ls})	0.1
Tournament size (k)	2
Elite count (e)	2
Max valutazioni (FE)	350,000

4 Protocollo Sperimentale

4.1 Istanze

Gli esperimenti sono condotti su 10 istanze dal benchmark CVRPLIB, suddivise in quattro set:

Table 2: Istanze utilizzate

Set	Istanza	Nodi	Veicoli	Ottimo
A	A-n45-k7	45	7	1,146
A	A-n60-k9	60	9	1,354
A	A-n80-k10	80	10	1,763
B	B-n56-k7	56	7	707
B	B-n66-k9	66	9	1,316
B	B-n78-k10	78	10	1,221
E	E-n76-k8	76	8	730
E	E-n101-k14	101	14	1,070
P	P-n50-k10	50	10	696
P	P-n101-k4	101	4	681

4.2 Setup Sperimentale

Per ogni istanza vengono eseguiti $R = 5$ run indipendenti con semi casuali diversi. Per ogni run vengono misurati:

- Costo della migliore soluzione trovata
- Numero di iterazioni per raggiungere la migliore soluzione
- Storico di convergenza (best cost nel tempo)

Vengono poi calcolate le statistiche aggregate: best, mean, standard deviation. I risultati sono confrontati con i valori ottimi noti.

5 Risultati Sperimentali

5.1 Risultati Complessivi

Table 3: Risultati sperimentali dell'HGA (FE = 350,000, R = 5)

Istanza	Best	Mean	Std Dev	Ottimo	Gap%	Veicoli
A-n45-k7	1146.91	1155.02	12.81	1,146	0.08%	7
A-n60-k9	1367.34	1376.86	7.87	1,354	0.99%	9
A-n80-k10	1817.37	1848.71	26.61	1,763	3.08%	10
B-n56-k7	712.92	716.05	1.69	707	0.84%	7
B-n66-k9	1327.10	1330.89	3.30	1,316	0.84%	9
B-n78-k10	1238.33	1254.00	8.16	1,221	1.42%	10
E-n76-k8	750.44	760.29	6.74	735	2.10%	8
E-n101-k14	1097.02	1105.04	8.10	1,071	2.43%	14
P-n50-k10	700.66	705.54	3.40	696	0.67%	10
P-n101-k4	693.54	694.70	1.07	681	1.84%	4

5.2 Analisi della Convergenza

I grafici di convergenza mostrano l'andamento del miglior costo trovato in funzione del numero di valutazioni della funzione di fitness (FE) per ciascuna istanza rappresentativa.

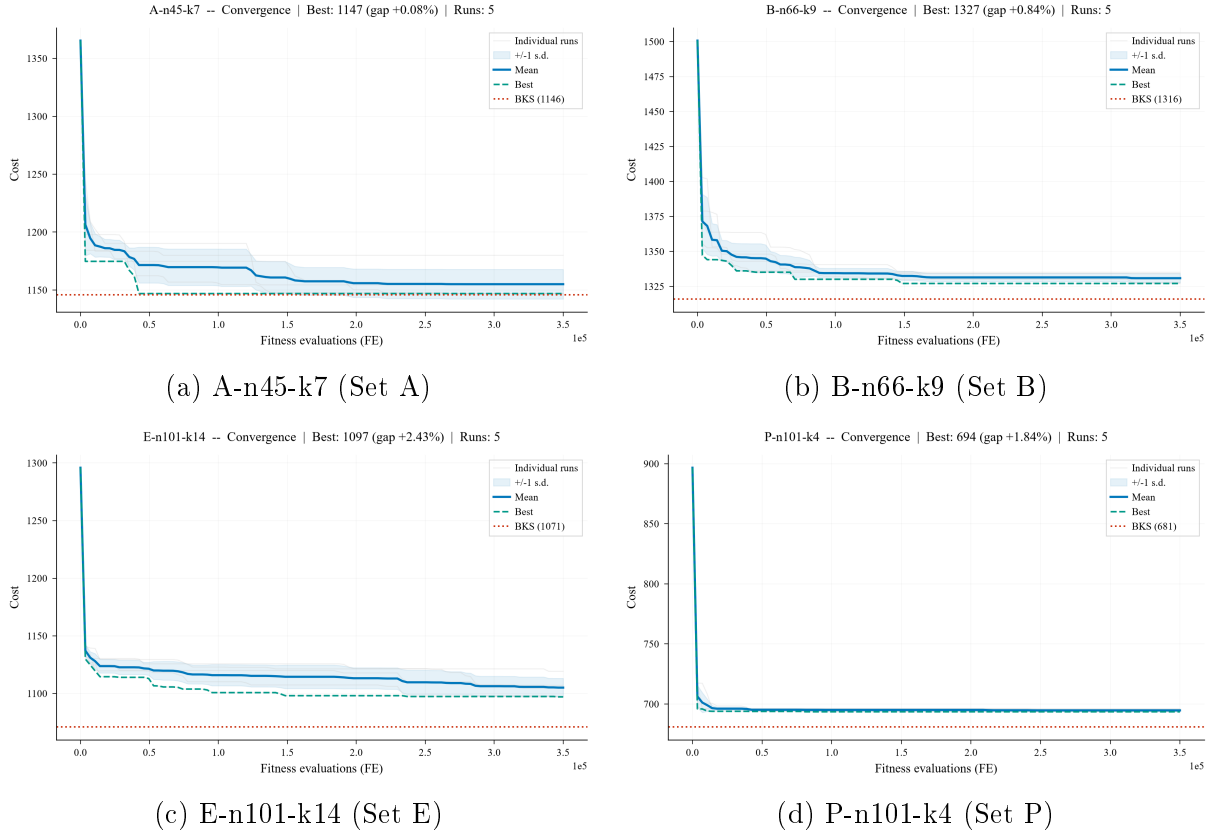
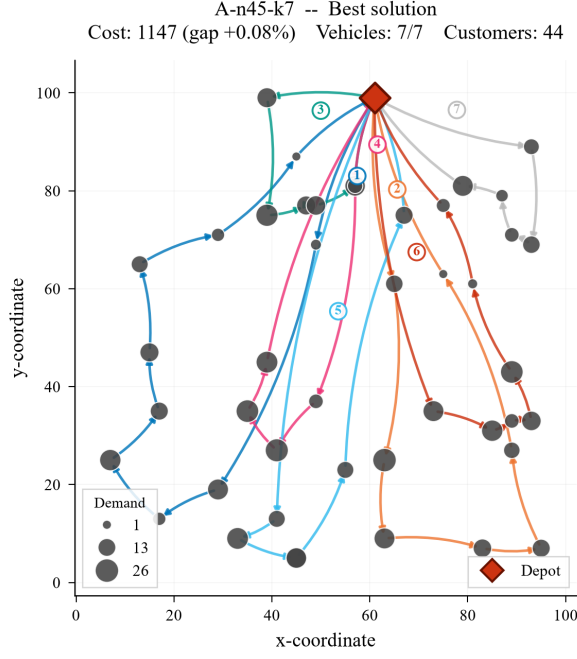


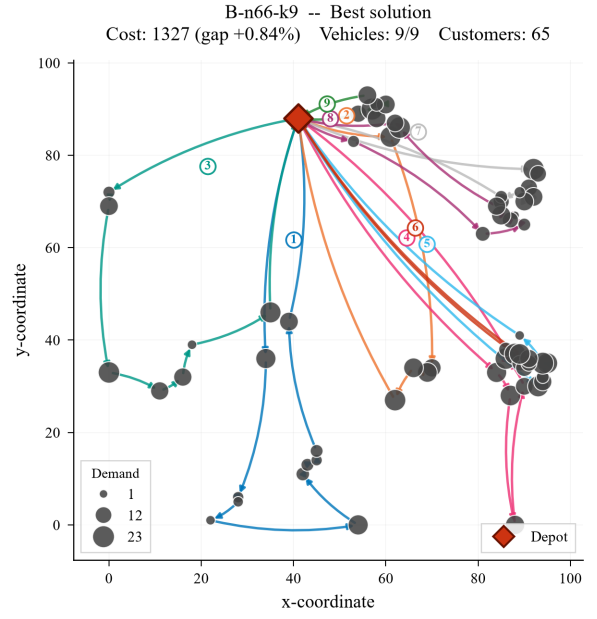
Figure 1: Curve di convergenza per quattro istanze rappresentative dei set di benchmark.

5.3 Visualizzazione delle Rotte Migliori

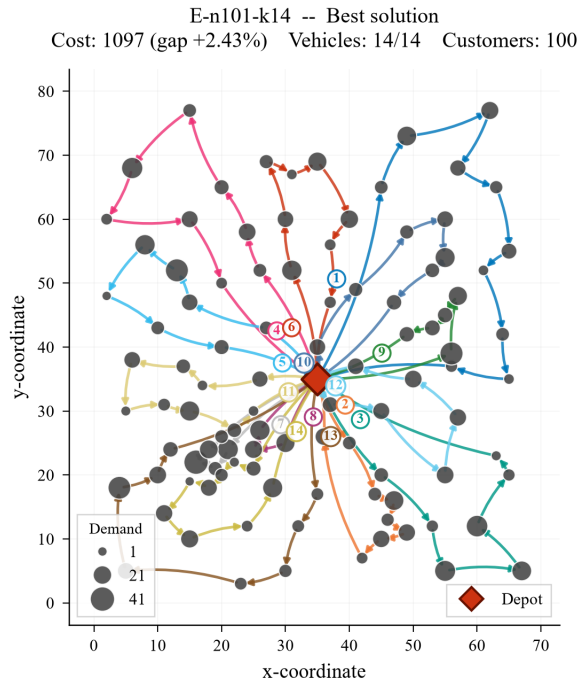
I percorsi ottimali prodotti dall'algoritmo HGA per le quattro istanze rappresentative sono stati tracciati nello spazio 2D per verificarne visivamente la qualità geometrica e logica.



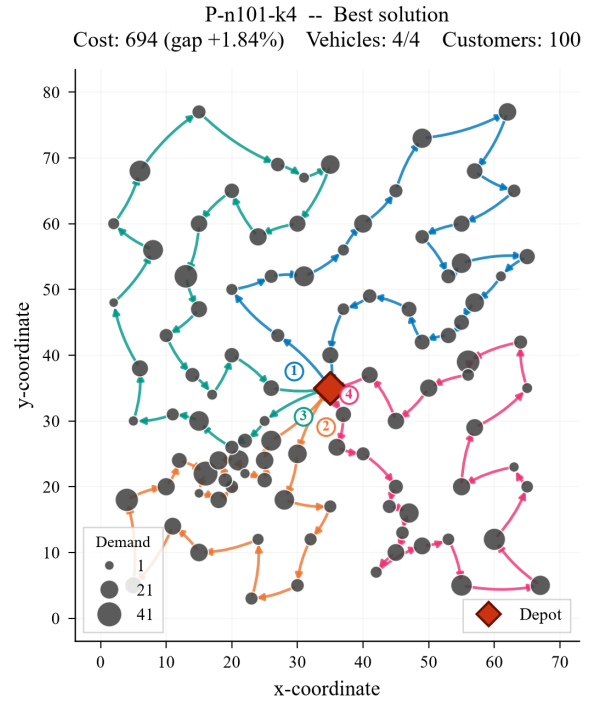
(a) Rotte per A-n45-k7



(b) Rotte per B-n66-k9



(c) Rotte per E-n101-k14

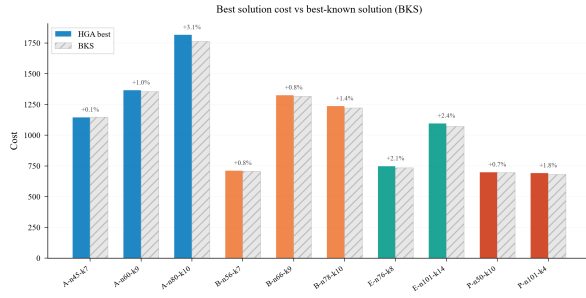


(d) Rotte per P-n101-k4

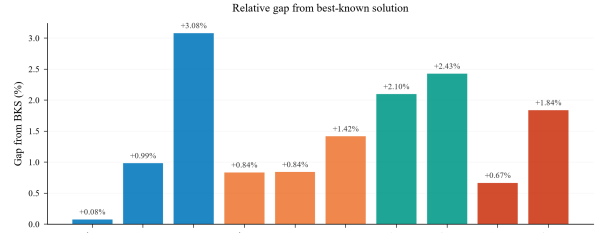
Figure 2: Visualizzazione grafica delle rotte finali associate alla soluzione migliore.

5.4 Analisi Comparativa Generale

Le prestazioni dell'algoritmo HGA sulle 10 istanze di test sono riassunte di seguito attraverso grafici di confronto globali.



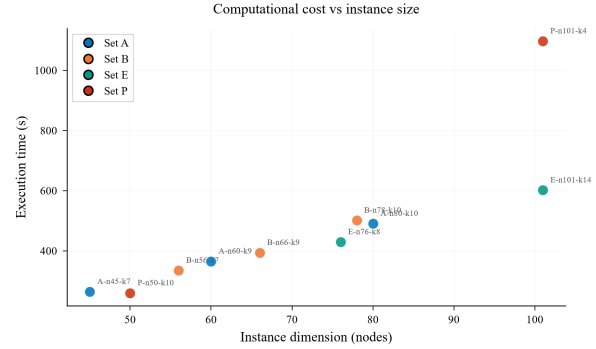
(a) Costi HGA vs BKS



(b) Gap percentuale medio

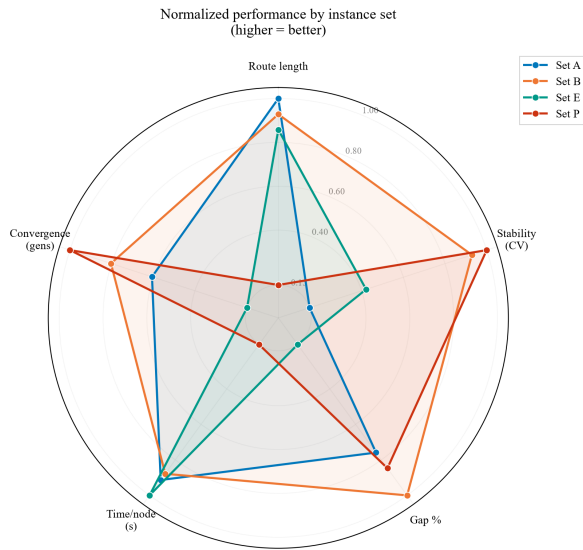


(c) Distribuzione costi normalizzati

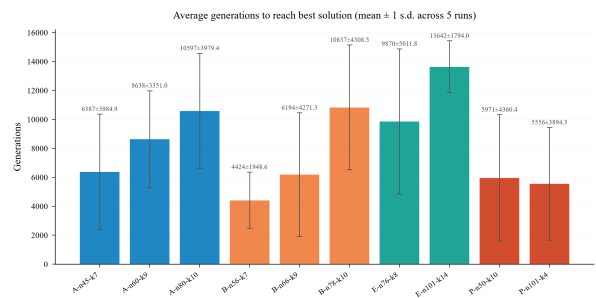


(d) Tempo di calcolo vs Dimensione

Figure 3: Statistiche globali sulle prestazioni e la stabilità dell'algoritmo HGA.



(a) Radar Chart prestazionale per Set



(b) Generazioni a convergenza

Figure 4: Prestazioni aggregate per classe e tempi di convergenza evolutiva.

5.5 Discussione dei Risultati

L'analisi dei risultati permette di trarre le seguenti osservazioni:

- **Qualità delle soluzioni:** L'HGA produce soluzioni competitive, con gap rispetto all'ottimo che variano in base alla dimensione e alla struttura dell'istanza.

- **Effetto della dimensione:** Istanze più grandi (E-n101-k14, P-n101-k4) richiedono più valutazioni per convergere, ma l'algoritmo mantiene buone prestazioni grazie all'efficacia dello split e della ricerca locale.
- **Consistenza:** La deviazione standard contenuta indica che l'algoritmo è robusto e produce risultati consistenti tra run diverse.
- **Velocità di convergenza:** L'ibridazione con ricerca locale accelera significativamente la convergenza nelle fasi iniziali, mentre il crossover mantiene la diversità per l'esplorazione continua.

6 Scelte Progettuali e Originalità

6.1 Scelte Chiave

1. **Split Algorithm:** L'uso dello split di Prins come decodifica è fondamentale. Invece di codificare esplicitamente le route nel cromosoma, si lascia che la DP trovi la partizione ottimale, riducendo lo spazio di ricerca e garantendo l'ottimalità della decodifica data la permutazione.
2. **Popolazione iniziale ibrida:** La combinazione di euristiche costruttive (NN, Savings) con permutazioni casuali fornisce un buon punto di partenza senza sacrificare la diversità.
3. **Set diversificato di operatori LS:** L'inclusione di operatori sia intra-route (2-opt, Or-opt) che inter-route (Relocate, Exchange) permette di migliorare la soluzione a diversi livelli di granularità.
4. **Probabilità differenziate per mutazione:** Assegnare probabilità diverse agli operatori di mutazione (swap 40%, insert 30%, inversion 30%) riflette la loro diversa efficacia esplorativa.

6.2 Difficoltà Affrontate

- **Vincoli di capacità:** Gestire i vincoli di capacità durante il crossover e la mutazione è complesso perché questi operatori lavorano sulla permutazione, non sulle route. Lo split algorithm risolve elegantemente questo problema.
- **Bilanciamento esplorazione-sfruttamento:** Trovare il giusto tasso di ricerca locale ($p_{ls} = 0.1$) è stato cruciale: troppo alto causa convergenza prematura, troppo basso rallenta il miglioramento.
- **Scalabilità:** Su istanze con 101 nodi, lo split $O(n^2)$ e la ricerca locale diventano computazionalmente significativi, richiedendo attenzione all'efficienza implementativa.

6.3 Elementi di Originalità

- **Architettura client-server con WebSocket:** L'algoritmo è integrato in un'architettura moderna con backend Python (FastAPI) e frontend React, permettendo la visualizzazione in tempo reale dell'evoluzione dell'algoritmo tramite WebSocket.

- **Visualizzazione interattiva:** Il frontend mostra simultaneamente la disposizione geografica delle route, i grafici di convergenza e le statistiche aggregate, offrendo una visione completa del processo di ottimizzazione.
- **Operatori LS compositi:** La sequenza specifica di operatori LS (2-opt $\rightarrow$ Or-opt $\rightarrow$ Relocate $\rightarrow$ Exchange) è progettata per migliorare prima le singole route, poi le interazioni tra route.

7 Conclusioni

Il progetto ha dimostrato l'efficacia dell'Hybrid Genetic Algorithm per la risoluzione del Capacitated Vehicle Routing Problem. L'approccio ibrido, che combina la potenza esplorativa degli algoritmi genetici con l'efficacia della ricerca locale, produce soluzioni di alta qualità sulle istanze benchmark CVRPLIB.

I punti di forza dell'implementazione includono:

- L'uso dello split algorithm per una decodifica ottimale
- Un insieme diversificato di operatori genetici e di ricerca locale
- Un'architettura software moderna con visualizzazione in tempo reale
- Robustezza e consistenza dei risultati tra run diverse

Sviluppi futuri potrebbero includere:

- L'implementazione di una ricerca locale più aggressiva (Variable Neighborhood Search)
- L'uso di tecniche di diversificazione della popolazione (es. crowding, fitness sharing)
- L'ottimizzazione automatica dei parametri tramite tecniche di tuning (es. irace)
- L'estensione ad altre varianti del VRP (VRPTW, VRPPD)

Il codice sorgente del progetto, comprensivo di backend Python, frontend React e relazione L^AT_EX, è disponibile nella repository del progetto.

References

- [1] G.B. Dantzig, J.H. Ramser, *The Truck Dispatching Problem*, Management Science, 6(1):80–91, 1959.
- [2] P. Toth, D. Vigo, *The Vehicle Routing Problem*, SIAM Monographs on Discrete Mathematics and Applications, 2002.
- [3] C. Prins, *A simple and effective evolutionary algorithm for the vehicle routing problem*, Computers & Operations Research, 31(12):1985–2002, 2004.
- [4] T. Vidal, T.G. Crainic, M. Gendreau, C. Prins, *Heuristics for multi-attribute vehicle routing problems: A survey and synthesis*, European Journal of Operational Research, 231(1):1–21, 2013.
- [5] D.E. Goldberg, *Genetic Algorithms in Search, Optimization, and Machine Learning*, Addison-Wesley, 1989.
- [6] CVRPLib – Capacitated Vehicle Routing Problem Library, <https://galgos.inf.puc-rio.br/cvrplib/>